EC-422 ARTIFICIAL INTELLIGENCE · COURSE PROJECT REPORT

AgriSense

Smart irrigation with reinforcement learning

Domain: Agriculture (intelligent irrigation, crop monitoring and resource optimisation)

Submitted by

Dushyant Kumar · 23145030

Nirmit Patil · 23145053

November 2026

Resource	Link
Website (live demo)	https://nirmit10000.github.io/agrisense/
Code	https://github.com/nirmit10000/agrisense
Presentation	https://www.canva.com/design/DAHaf0-Y1zI/9xi2UGtj7G3lCuj4Jpl6Uw/view

Summary

Most farmers irrigate on a fixed calendar that ignores rain and the crop's actual need. We framed the daily irrigation decision as a reinforcement-learning problem and trained a tabular Q-learning agent in a field simulator built on the FAO-56 soil-water balance and driven by 16 years of real daily weather from three contrasting Indian climates (Pune, Jodhpur, Ludhiana). Each day the agent sees soil dryness, crop stage, a 3-day rain forecast and evaporative demand, and chooses 0-40 mm of water. On 60 test seasons (2021-2024) that it never saw in training, across maize, wheat and tomato, it used 45% less water than a fixed schedule with higher yield (98.1% vs 93.7% of maximum), and 5.5% less water than the FAO soil-moisture threshold rule, giving the best water productivity of all policies (6.30 kg/m³). The learned policy is a 360-row table that can be read as a rulebook, and everything is shown on a public website.

1. Problem and objective

Agriculture uses most of India's fresh water, and much of it is wasted: water applied just before rain, or more than the root zone can hold, drains below the roots and is lost. Too little water in a hot spell costs yield instead. Deciding when and how much to irrigate is therefore a trade-off made every day under uncertainty (future rain and heat), and each decision changes the soil moisture for the next one. This is the setting reinforcement learning (RL) is designed for.

Objective: learn an irrigation policy that minimises water use and waste while keeping yield close to its potential, works across crops and climates, and can be explained; then compare it fairly with the strategies farmers and agronomists use today.

2. Data

Daily weather for 1 Jan 2010 - 31 Dec 2025 was downloaded from the free Open-Meteo Historical Weather API (ERA5 reanalysis): rainfall, FAO Penman-Monteith reference evapotranspiration (ET0) and max/min temperature. Three locations were chosen for maximum climatic contrast, so the agent has to generalise rather than memorise one place. The whole dataset is three CSV files of about 0.5 MB.

City	Climate	Mean rain (mm/yr)	Range (mm)	ET0 (mm/yr)
Pune	Semi-arid tropical	1122	681-1485	1626
Jodhpur	Arid (Thar desert)	432	172-759	2059
Ludhiana	Sub-tropical (Punjab breadbasket)	783	511-1162	1493

Crops and sowing seasons: Maize: Kharif (monsoon), Zaid (summer); Wheat: Rabi (winter); Tomato: Rabi (winter), Zaid (summer). Train/test split by time: seasons starting 2010-2020 are used for training; seasons starting 2021-2024 are held out and never seen by the agent. Splitting by year rather than at random is the honest choice for weather data, because neighbouring days are correlated.

3. Method



3.1 Field simulator (the RL environment)

The simulator advances one day at a time with the FAO-56 root-zone water balance. Dr is the root-zone depletion (mm below field capacity), TAW the total water the root zone can hold and RAW = p × TAW the part the crop can use without stress. Each day:

- Rain and irrigation enter the soil: Dr ← Dr − P − I. Anything that would push Dr below 0 drains below the roots and is lost.
- Water stress: Ks = 1 if Dr ≤ RAW, otherwise Ks = (TAW − Dr) / (TAW − RAW).
- Crop water use: ETc = Kc(day) × ET0, ETa = Ks × ETc, then Dr ← min(TAW, Dr + ETa).

At the end of the season, yield follows the FAO-33 yield response:

$$1 - Ya / Ym = Ky \cdot (1 - \Sigma ETa / \Sigma ETc)$$

Crop parameters come from the FAO-56 and FAO-33 tables; the soil is a sandy loam (field capacity 0.22, wilting point 0.10 m^3/m^3).

Crop	Stage lengths (days)	Season (days)	Kc ini / mid / end	Root depth (m)	p	Ky
Maize	20 / 35 / 40 / 30	125	0.3 / 1.2 / 0.6	0.8	0.55	1.25
Wheat	20 / 25 / 60 / 30	135	0.3 / 1.15 / 0.4	0.8	0.55	1.15
Tomato	30 / 40 / 45 / 30	145	0.6 / 1.15 / 0.8	0.6	0.4	1.05

3.2 Markov decision process

Element	Definition
State (360)	Soil dryness Dr/TAW (10 levels) × crop stage (4) × rain forecast for the next 3 days (<2, 2-15, >15 mm) × today's ET0 (<3, 3-5, >5 mm)
Action (5)	Irrigate 0, 10, 20, 30, 40 mm today
Reward (daily)	−(1·irrigation + 10·(ETc − ETa) + 1·drained irrigation + 10 per irrigation event)
Episode	One crop season (125-145 days) from a random training year, city and initial soil moisture
Forecast	True rain of the next 3 days with ±25 % random error (useful but imperfect)

The state holds exactly what a farmer would look at, and is small enough for a table. The key design choice is the stress weight: one mm of unmet crop demand costs 10× one mm of irrigation, because lost yield matters far more than water. The per-event cost stops the agent from giving many tiny doses that no farmer would apply.

3.3 Q-learning

Q(s, a) estimates the long-term reward of action a in state s. After every simulated day it is updated with

$$Q(s,a) \leftarrow Q(s,a) + \alpha \cdot [\, r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a) \,]$$

Actions are chosen ε-greedily: with probability ε a random action (exploration), otherwise the best known one. One agent is trained per crop; every episode picks a random city, so one agent must work in all three climates.

Hyper-parameter	Value
Training episodes	50,000 seasons per crop
Learning rate α	0.1 → 0.01 (linear decay)
Discount factor γ	0.95
Exploration ε	1.0 → 0.05 over the first 70 % of episodes
Random seed	42 (training is fully reproducible)
Training time	about 1 minute per crop (3.5 min for all three) on a laptop CPU

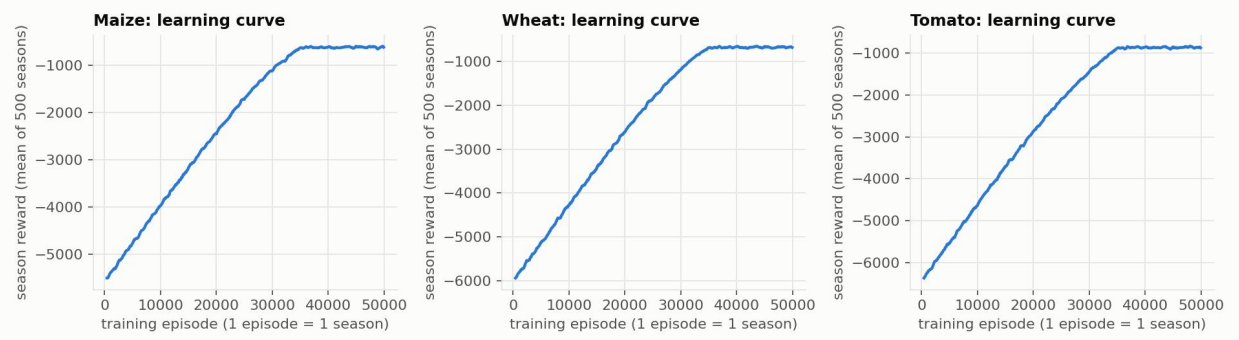


Figure 1. Average season reward during training: it rises while exploration decays, then plateaus once the agent has converged.

3.4 Baselines and evaluation

- **Rainfed:** never irrigate (lower bound).
- **Fixed schedule:** 40 mm every 7 days, a typical farmer calendar.
- **Sensor threshold rule:** when depletion passes RAW, refill the root zone to field capacity. This is the FAO-recommended rule and a strong expert baseline: it gets perfect soil-moisture readings and any amount of water, but it does not use the forecast.

Every policy is run on every combination of crop, sowing season, city and test year 2021-2024 (60 seasons per policy), with identical weather and identical forecast errors.

4. Results

Policy (average of 60 test seasons)	Water (mm)	Irrigations	Wasted (mm)	Stress days	Yield	kg / m³
Q-learning agent (RL)	**435**	**13.8**	**1.6**	**12.4**	**98.1 %**	**6.30**
Sensor threshold rule	460	11.4	4.2	0.0	100.0 %	6.13
Fixed schedule	784	19.6	354.0	15.5	93.7 %	3.33
Rainfed (no irrigation)	0	0.0	0.0	89.6	31.8 %	-

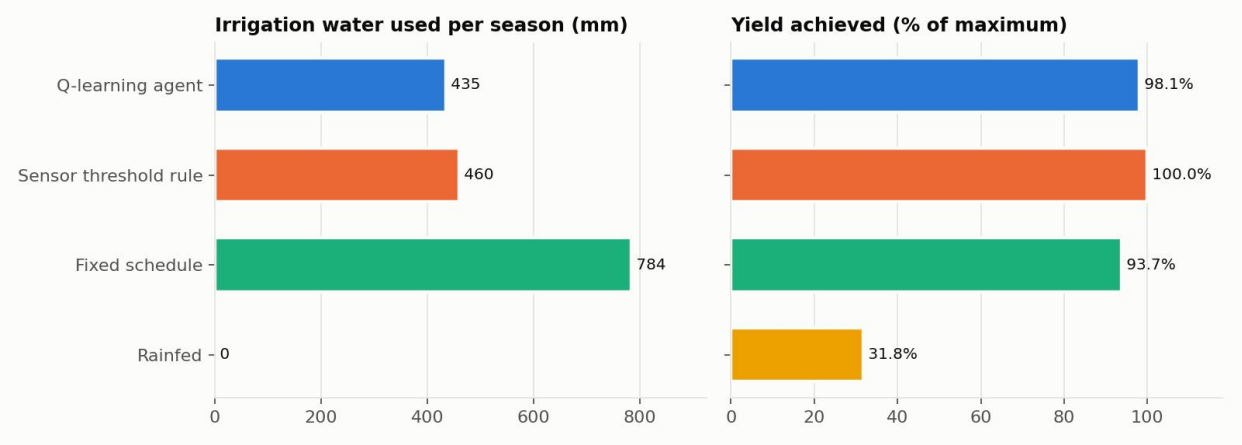


Figure 2. Irrigation water per season and yield, averaged over 60 unseen test seasons.

The agent used 435 mm per season against 784 mm for the fixed schedule (45% less) and raised yield from 93.7% to 98.1%: the calendar drains about 354 mm per season because it waters regardless of rain, and still causes stress in hot spells. Against the expert threshold rule it used 5.5% less water and lost less to drainage (1.6 vs 4.2 mm) for a 1.9-point lower yield, giving the best water productivity (6.30 vs 6.13 kg/m³).

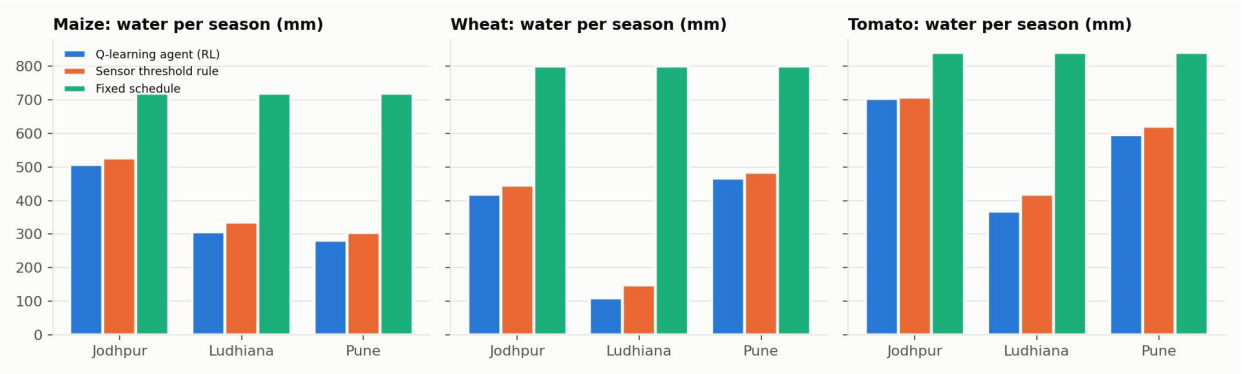


Figure 3. Irrigation per season by crop and city. Water needs differ about threefold across climates; the agent adapts without being told which city it is in.

City	RL water (mm)	RL yield	Rule water (mm)	Rule yield	Fixed water (mm)	RL saving vs rule
Pune	444	99.5 %	466	100.0 %	784	4.7 %
Jodhpur	568	99.5 %	582	100.0 %	784	2.4 %
Ludhiana	292	95.4 %	331	100.0 %	784	11.9 %

Weak spot: in Ludhiana the agent saves the most water but loses the most yield (95.4% of maximum). The likely cause is the long, cold winters with very low ET0, where the coarse discrete state hides small differences and the agent sometimes waits too long. In arid Jodhpur, with little rain to exploit, it behaves almost exactly like the threshold rule.



Figure 4. One test season day by day (maize, Jodhpur, Kharif 2023). The sensor rule refills the root zone as soon as the stress line is reached; the agent lets the soil dry a little further and gives smaller doses: 310 mm instead of 385 mm, for 98.9% instead of 100.0% yield.

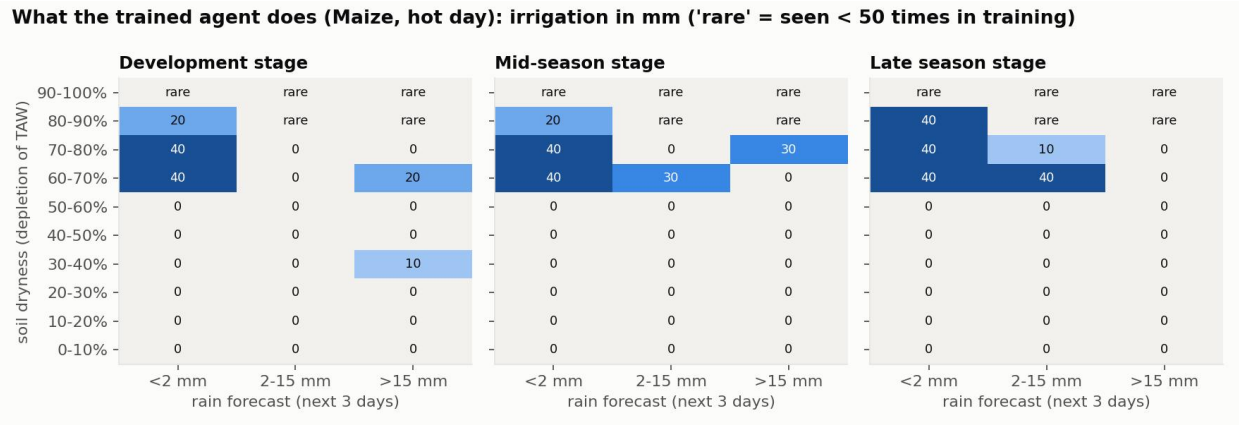


Figure 5. The learned policy for maize on a hot day: irrigation (mm) for each soil dryness and rain forecast, in three growth stages.

Because the policy is a table, it reads like a rulebook. Without being given any agronomic rule, the agent found that it should:

- not irrigate until about 60% of the available soil water is used (the FAO-56 threshold for maize is 55%);
- give the full 40 mm when the soil is dry and no rain is forecast;
- irrigate less or not at all when heavy rain is forecast. This is where it beats the threshold rule, which cannot see the forecast.

5. Website

The results are deployed as a public static website on GitHub Pages (https://nirmit10000.github.io/agrisense/), built with plain HTML, CSS and JavaScript (charts with Apache ECharts), with no server. All 225 seasons (every city, crop, sowing season and year 2010-2024) are simulated for all 4 policies in Python with the trained Q-tables and exported to one JSON file, so the site shows exactly the numbers in this report. The Overview tab summarises the method; the Smart irrigation tab has a season simulator (crop, city, season, year, policies, day-by-day replay), the learned rulebook for any crop stage and evaporative demand, and the test results.

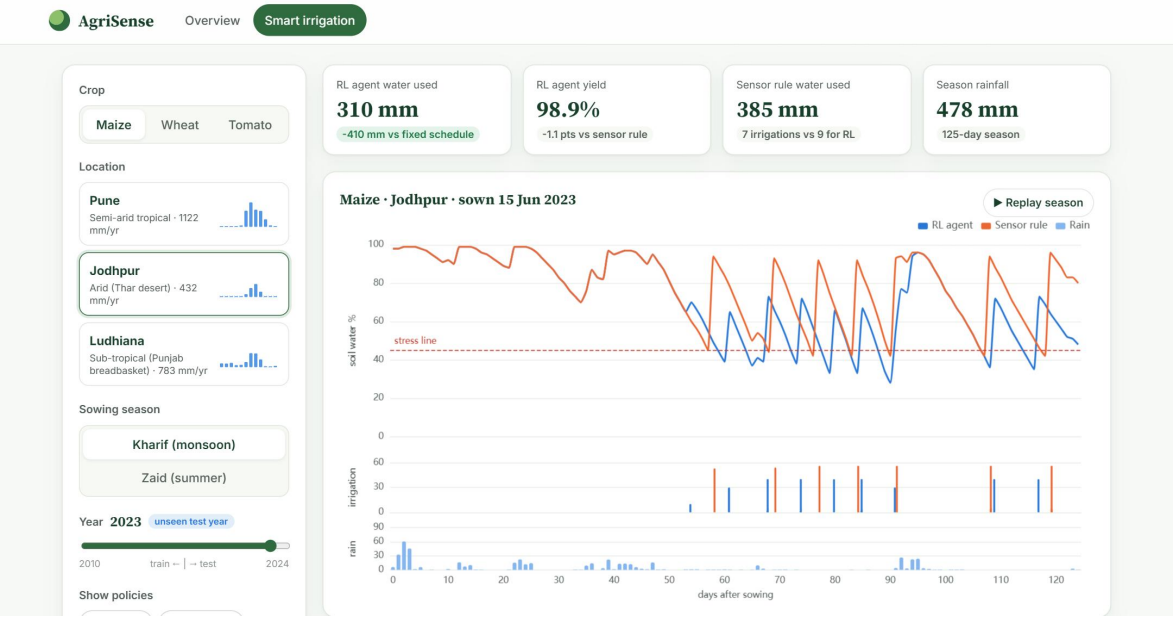


Figure 6. Season simulator on the website: soil water, irrigation and rain day by day for the RL agent and the sensor rule.

6. Limitations and future scope

Limitations

- **Simulated field:** results come from an FAO-56 model, not a field trial; real soils vary and sensors are noisy.
- **Simple yield model:** the seasonal FAO-33 equation ignores that stress at flowering hurts more.
- **Synthetic forecast:** true rain ± 25% error, not archived real forecasts.
- **Coarse state and one soil:** 10 dryness and 3 weather levels lose detail (Ludhiana winters); only sandy loam was trained.

Future scope

- **Deep Q-Network:** continuous inputs instead of the 360-state table (the simulator already returns them).

- **Real sensors and live forecasts:** IoT soil-moisture readings plus a weather API for a daily recommendation.
- **Soil and crop in the state:** one agent for every field instead of one per crop.
- **Stage-sensitive crop model:** e.g. AquaCrop, so stress at flowering counts more.

7. Code and how to run

Path	Contents
agrisense/config.py	Cities, crops (FAO parameters), soil, reward weights, training settings, file paths
agrisense/weather.py	Downloads and caches the Open-Meteo weather data
agrisense/env.py	The field simulator (FAO-56 water balance, FAO-33 yield)
agrisense/agents.py	The three baseline policies, the state encoder and the Q-learning agent
agrisense/train.py	Trains one Q-learning agent per crop
agrisense/evaluate.py	Runs all policies on the test seasons; writes results, figures and the website data
data/	The three weather CSV files (2010-2025)
models/	The trained Q-tables (q_maize.npz, q_wheat.npz, q_tomato.npz)
results/	Test results (CSV) and figures
index.html, web/	The website (web/data/irrigation.json is produced by evaluate.py)

```
pip install -r requirements.txt
python -m agrisense.train       # ~3.5 min; seed 42 reproduces the models exactly
python -m agrisense.evaluate    # test results, figures and web/data/irrigation.json
python -m agrisense.weather     # optional: download the weather again
python -m http.server           # then open http://localhost:8000
```

References

[1] Allen, R. G., Pereira, L. S., Raes, D., & Smith, M. (1998). Crop evapotranspiration: Guidelines for computing crop water requirements. FAO Irrigation and Drainage Paper 56. FAO, Rome.

[2] Doorenbos, J., & Kassam, A. H. (1979). Yield response to water. FAO Irrigation and Drainage Paper 33. FAO, Rome.

[3] Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. Machine Learning, 8(3-4), 279-292.

[4] Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press.

[5] Zippenfenig, P. (2023). Open-Meteo.com Weather API. https://open-meteo.com (Historical Weather API).

[6] Hersbach, H., et al. (2020). The ERA5 global reanalysis. Quarterly Journal of the Royal Meteorological Society, 146(730), 1999-2049.